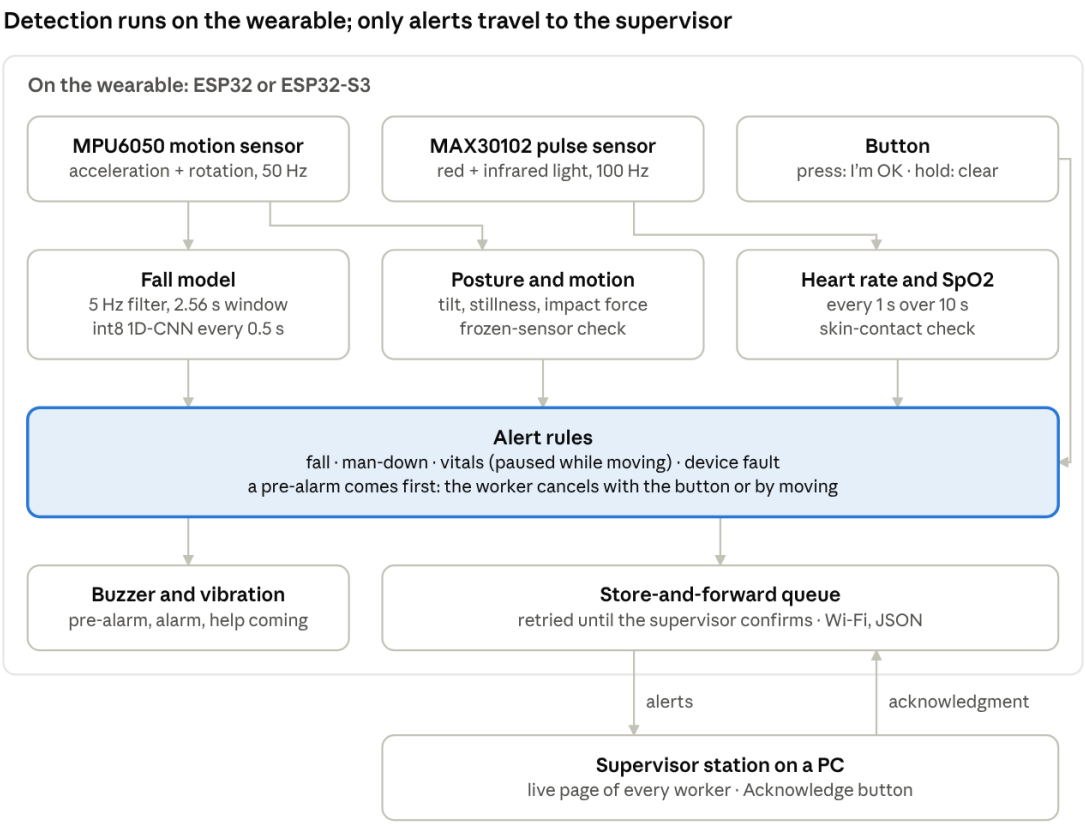
Edge-AI Worker Safety Monitor — Project Report

Nov 8, 2026 · @Chaithr

A waist-worn ESP32 wearable detects falls on the device with an 18.6 kB int8 neural network, catches collapses and abnormal vitals, and alerts a supervisor over Wi-Fi. On 6 unseen KFall subjects it detected 441 of 441 falls with 1 false alarm in 516 activity recordings, raising the alarm a median 0.81 s after impact. The firmware runs bit-identical to the Python model in the ESP32 emulator and is tested automatically on every push; it has not yet run on real hardware.

System at a glance



System overview · sensors, on-device processing, alert rules, supervisor

Raw sensor data never leaves the device: the wearable decides, and only alerts, status and acknowledgments travel over Wi-Fi.

Goal and scope

The device must tell a supervisor within seconds when a worker falls, collapses or shows abnormal vitals, with all detection running on the wearable itself. The work is split into two phases.

Phase	What it delivers	Status
Phase 1: model	Training pipeline on the KFall dataset, an 18.6 kB int8 1D-CNN, a C header the firmware compiles in, metrics and figures	Done
Phase 2: firmware	ESP32 / ESP32-S3 application, alert rules, supervisor station, automated tests, live demo	Software done and tested in the emulator; not yet run on a board

Hardware target: an ESP32 or ESP32-S3, an MPU6050 accelerometer and gyroscope worn at the lower back, a MAX30102 pulse oximeter, a buzzer, a vibration motor and one button. Alerts go over Wi-Fi to a supervisor station on a PC; a LoRa link is planned but not built.

Phase 1 pipeline

`phase1_pipeline.py` turns raw KFall recordings into a firmware-ready int8 model in one run, using only signals the MPU6050 can actually produce and only processing the ESP32 can repeat sample by sample.

Step	Choice	Why
Dataset	KFall: 32 subjects, 21 activity types, 15 fall types, lower-back IMU at 100 Hz, expert-labelled impact frames; 5,057 usable recordings (2,346 falls)	Sensor placed where the device will be worn; labelled impacts allow fall-aligned windows
Channels	Acceleration x, y, z (g) and angular rate x, y, z (deg/s); Euler angles dropped	The MPU6050 has no fused orientation on the chip
Pre-processing	Resample to 50 Hz, causal 4th-order Butterworth low-pass at 5 Hz, state initialised from the first sample	Exactly what the firmware runs; a causal filter needs no future samples
Windows	128 samples (2.56 s), a new decision every 25 samples (0.5 s); fall windows placed around the impact during training	Short enough to react fast, long enough to see the fall and the aftermath
Split	By subject: 22 train, 4 validation, 6 test subjects never seen in training	Measures how the model does on new people
Augmentation	Random ±10° mounting rotation, ±8 % gain, sensor noise	Robustness to how the device is clipped on

Step	Choice	Why
Model	1D-CNN: three convolution blocks (16, 32, 32 filters), global average pooling, dense 16, sigmoid; 7,169 parameters	Uses only TensorFlow Lite Micro-friendly ops (no LSTM)
Decision	Alarm when 2 consecutive windows score at or above the threshold (now 0.50)	One noisy window cannot raise an alarm
Quantisation	Full int8 (weights, activations, input and output)	18.6 kB model, fast integer inference on the ESP32
Export	`fall_model_data.h`: model bytes, normalisation, filter coefficients, quantisation scales, threshold and the list of ops to register	The firmware takes every constant from this one file

The pipeline also contains reference algorithms for heart rate and SpO2 (band-pass filter, peak detection, ratio of ratios) and the original alert state machine; both were ported to C in Phase 2.

Phase 1 results

The final int8 model detects every fall of the 6 test subjects (441 of 441) with 1 false alarm in 516 normal-activity recordings, and two fixes to the training pipeline cut false alarms from 5 to 1.

Version	Falls detected (test)	False alarms (of 516 recordings)	Per-recording accuracy	5-fold CV specificity
Final: fixed early stopping, threshold 0.50	441 / 441	1	99.90 %	99.86 ± 0.28 %
Fixed early stopping, threshold 0.05	441 / 441	2	99.79 %	99.76 ± 0.34 %
Original upload (Oct 2026)	441 / 441	5	99.48 %	97.19 ± 3.01 %

Final model in detail (int8, the one on the device): per window 99.93 % accuracy, 100 % sensitivity, 99.92 % specificity, F1 99.77 %; per recording 99.90 % accuracy, 100 % sensitivity, 99.81 % specificity, F1 99.89 %. Five-fold subject-wise cross-validation gives per-recording sensitivity 100.00 ± 0.00 % and per-window accuracy 99.97 ± 0.05 %. The int8 model scores the same as the float32 model.

Fix 1, early stopping. Training stopped on validation AUC, which is already 1.000 after the first epoch, so it never improved and the saved model was rolled back to epoch 1. Stopping on validation loss instead trains 33 epochs.

Fix 2, threshold. On the validation subjects every threshold from 0.05 to 0.95 scores perfectly, and the old rule took the lowest. The pipeline now takes the middle of the tied range (0.50), the choice furthest from both failure modes. Re-running the pipeline reproduced a bit-identical model, so only the threshold changed.

Also fixed: `requirements.txt` lacked `pandas` and `openpyxl`, which loading KFall needs.

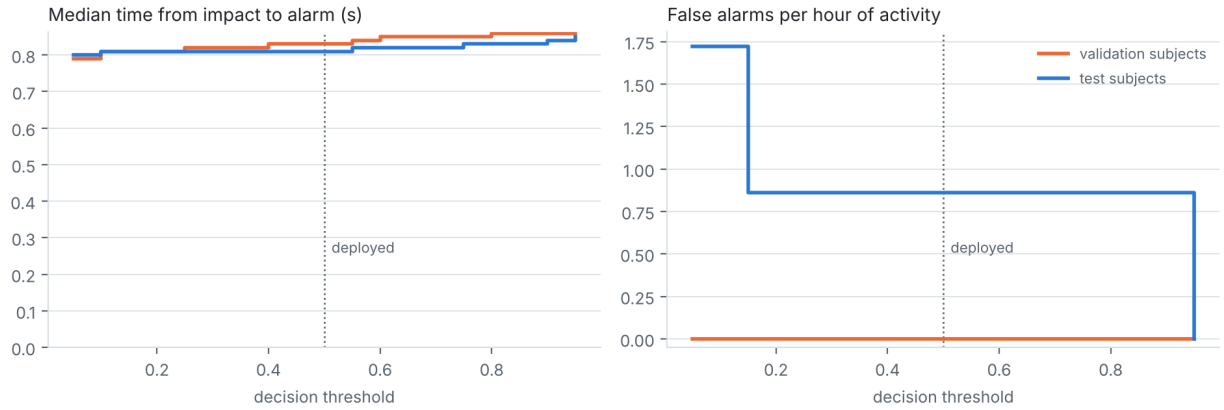
Device-level evaluation

Replayed exactly as the firmware decides, the model raises a suspected fall a median 0.81 s after impact and makes 0.9 false alarms per hour of KFall activity. `evaluate_deployment.py` slides the int8 model over every validation and test recording every 0.5 s and applies the 2-consecutive-windows rule; latency is the alarm time minus the labelled impact.

Subjects (threshold 0.50)	Falls detected	Median time to alarm	95th percentile	Alarm within 1 s	False alarms during normal activity
Test (6 subjects)	441 / 441	0.81 s	1.08 s	85 %	1 in 1.16 h: 0.9 per hour (95 % CI 0.0–4.8)
Validation (4 subjects)	297 / 297	0.83 s	1.08 s	84 %	0 in 0.82 h (95 % CI 0.0–4.5 per hour)

The single false alarm comes from KFall activity T05. KFall's activity clips are short and packed with hard movements, so this rate is a pessimistic bound for a normal shift, and with so few events the interval is wide. On the device a false alarm first opens a 10 s cancel window; it reaches the supervisor only if the worker neither presses "I'm OK" nor moves.

Every threshold from 0.05 to 0.95 detects all falls of both subject groups; raising it mainly removes false alarms and adds a few tens of milliseconds of latency. The threshold is chosen from the validation subjects only, never from the test curve.



Threshold sweep from 0.05 to 0.95 in steps of 0.05, test and validation subjects; the dotted line marks the deployed threshold, 0.50.

Phase 2 firmware

The ESP-IDF v5.5 application runs the Phase 1 model on an ESP32 or ESP32-S3 and takes every model constant from `fall_model_data.h`, so a retrained model only needs a rebuild.

Part	What it does
`safety_core` (portable C)	Low-pass filter, 128 × 6 sliding window, int8 quantisation, 2-window decision rule, posture, heart rate and SpO2, alert state machine. Builds for the ESP32 and on a PC, so the same code is tested against Python
MPU6050 driver	50 Hz from the on-chip FIFO (no drift, no lost samples), ±16 g and ±2000 deg/s, 20 Hz anti-aliasing filter, configurable axis mapping to the KFall frame
TensorFlow Lite Micro	Runs the int8 model with the 7 ops it needs; Espressif's esp-tflite-micro 1.4.x with optimised kernels
MAX30102 driver	Red and IR at 100 Hz, 18-bit; heart rate and SpO2 every second over a 10 s window; skin-contact check
Buzzer, vibration, button	Patterns for pre-alarm, alarm and "help is on the way"; short press = "I'm OK", 3 s press clears an alert
Alert link	Wi-Fi station, JSON over HTTP, store-and-forward queue with retries, acknowledgments from the supervisor
Supervisor station	`supervisor.py` on a PC: live page of every worker, alerts with an Acknowledge button, log file
Replay and demo mode	Feeds a recorded synthetic sensor stream through the real pipeline, for the emulator tests and the live demo

Measured on the current build (ESP32, ESP-IDF v5.5.6): firmware image 1,010,607 bytes, 64 % of the 1.5 MB app partition; static RAM 118 KB of 177 KB (65 %). The model's tensor arena is 16 KB; the emulator measured 7.3 KB used on the ESP32 and 10.3 KB on the ESP32-S3, whose vector kernels need more scratch space. Inference speed on a real chip is not yet measured; emulator timings are not real speeds.

Safety features

Beyond the Phase 1 fall rule, the device catches collapses without an impact, ignores vitals it cannot trust, reports its own sensor failures and never loses an alert. With man-down and vitals gating switched off, the state machine behaves exactly like the Phase 1 `SafetyFSM`; this is checked automatically.

Rule	Trigger	What happens	Why
Fall	2 consecutive windows with probability ≥ 0.50	Pre-alarm (beep + vibrate) for up to 10 s; ALERT_FALL if the worker stays still 5 s or does not press "I'm OK"	The Phase 1 rule, ported line for line
Man-down	Motion below 0.02 g for 30 s while lying	15 s pre-alarm; moving or the button cancels it; otherwise	Fainting, gas or heat stroke causes a

Rule	Trigger	What happens	Why
	(body tilted over 60°), or 180 s upright, while worn	ALERT_MAN_DOWN	collapse with no impact for the fall model to see
Vitals	Heart rate below 45 or above 140 bpm, or SpO2 below 90 %, for 10 s	ALERT_VITALS	Phase 1 limits
Motion-aware vitals	Mean motion over the last 10 s above 0.35 g	Heart rate and SpO2 are ignored until it calms	Running corrupts the PPG; in testing, jogging read a false 165 bpm
Skin contact	Mean IR level below 30,000 counts	Vitals shown as unavailable; man-down not armed	A loose sensor must not raise alarms
Sensor fault	100 identical motion samples in a row (2 s), or no data for 2 s	DEVICE_FAULT to the supervisor; DEVICE_OK when data returns	A silent failure of fall detection is the worst case
Store-and-forward	Supervisor unreachable	Up to 16 messages wait in a queue and are retried, back-off 1 s doubling to 30 s, until confirmed, in order	A Wi-Fi dropout delays an alert but never loses it
Acknowledgment	Supervisor clicks Acknowledge	The device switches to two short beeps every 3 s: help is on the way	The worker knows the alert got through

Every alert carries a unique ID, the worker ID, posture and tilt, motion level, heart rate, SpO2 and, for falls, the impact force (5.6 g in the test scenario). All limits are settings in `idf.py menuconfig`.

Verification without hardware

Every level of the system is checked against the Python reference, and all but the KFall check run automatically on every push in GitHub Actions; the latest run (#8) passed all seven jobs.

Level	What is checked	Result
1. C port vs Python, synthetic data (every push)	Low-pass filter against `scipy.signal.sosfilt`; int8 model inputs; 2-window rule; heart rate and SpO2 on 46 PPG recordings; state machine in 304 Phase 1 scenarios and 400 long scenarios with man-down, posture and motion	Filter within 0.000004 g and 0.0002 deg/s; 5 of 614,400 int8 inputs off by one; HR and SpO2 identical; 798 + 741 events, 0 mismatches

Level	What is checked	Result
2. C port on real KFall data (local; the KFall licence keeps it out of CI)	All 957 recordings of the 6 test subjects through the C pre-processing, then the model on the C code's windows	12 of 8,134,656 int8 inputs off by one; decisions identical to Python on every recording: 441 / 441 falls, 1 false alarm
3. Supervisor station (every push)	Acknowledgment round trip, duplicate retries, dashboard button, auto-acknowledge, online / offline, bad requests, HTML escaping	7 of 7 tests pass
4. Real firmware builds (every push)	ESP-IDF v5.5.6 for ESP32 and ESP32-S3, warnings treated as errors	Both build
5. ESP32 emulator, end to end (every push)	250 s scenario of 9 situations; events against the Python device model; model self-test; supervisor started 20 s late; acknowledgments	11 of 11 events at the same times as Python; 28 of 28 model outputs identical to TensorFlow's reference kernels; 8 of 8 messages delivered in order after retries; 4 of 4 alerts acknowledged
6. ESP32-S3 emulator (every push)	Same scenario with the S3's different optimised kernels, no network	11 of 11 events; 28 of 28 model outputs identical
7. Live demo script (every push)	`tools/demo.sh --ci`: the real-time demo, unattended	Passes

The 9 emulator situations: normal work; jogging with PPG artefacts; a fall with the worker lying still; a fall the worker cancels; sustained tachycardia; a slow collapse without impact; lying still then moving; standing still for 40 s; a frozen motion sensor. All test data committed to the repository is synthetic; KFall data stays local.

Live demo

The real firmware runs in the ESP32 emulator on a laptop and plays the 9 situations in real time (about 5 minutes) while the supervisor page in a browser receives the alerts; the audience acknowledges each alert by clicking, and the device answers. It needs only Docker, no board.

1. Once before the meeting (5–10 minutes, builds the firmware and installs the emulator): `docker run --name safety-demo -it -p 8080:8080 -v "$PWD":/project -w /project/firmware espressif/idf:v5.5.6 bash tools/demo.sh`
2. In the meeting: `docker start -ai safety-demo`, then open `http://localhost:8080/`.
3. Terminal (the worker's device) and browser (the supervisor) side by side. Each alert waits until someone clicks Acknowledge; the device then logs "help is on the way" and the story continues.

What the audience sees, in order: the worker online with heart rate; jogging with no false alarm ("vitals paused"); a fall alert with posture lying and impact 5.6 g; a fall cancelled on the device; a racing-heart alert; a man-down alert after a slow collapse; a man-down pre-alarm cancelled by moving; a sensor-fault alert. It ends with the model self-test (28 of 28 identical) and "REPLAY RESULT: PASS". `DEMO.md` in the repository has the minute-by-minute script, fallbacks and a real-hardware variant; CI runs the demo unattended on every push.

Limitations and next steps

Nothing has run on a real board yet, and the alert limits and SpO2 have not been checked on people; those are the next steps, in roughly this order.

- ☐ Wire an ESP32 with the MPU6050 and MAX30102, flash it, and confirm the sensor axes match KFall (standing still: posture upright, tilt near 0°)
- ☐ Measure on the chip: inference time, RAM in use, battery life over a shift
- ☐ Controlled falls onto a mattress while wearing the device, with someone present, to confirm detection end to end
- ☐ Calibrate SpO2 against a reference oximeter; the 110 − 25·R curve is uncalibrated, and some MAX30102 boards swap the red and IR LEDs
- ☐ Tune the man-down times (30 s lying, 180 s upright), stillness level (0.02 g) and vitals motion limit (0.35 g) on real workers
- ☐ Wear trials at work to measure false alarms per hour of a real shift (the 0.9 per hour above comes from dense KFall activity clips)
- ☐ Validation subjects that are harder for the model, so the threshold is driven by data rather than the middle of a perfect plateau
- ☐ LoRa as a second alert link for sites without Wi-Fi (needs a radio module and driver)
- ☐ Keep queued alerts across a reboot (today the queue lives in RAM)

The heart-rate, SpO2 and alert rules are checked against synthetic signals only and are not clinically validated.

Reproducing the work

Everything is on branch `claude/great-euler-gnc1oj` of chaithr45/Snehitha_Proj; each result above can be rerun with one command from the repository root.

Result	Command	Time
Train and evaluate the model (writes `fall_model_data.h`, `results.md`, figures)	`python phase1_pipeline.py --data kfall --cv 5`	about 20 min on 4 CPU cores
Device-level evaluation	`python evaluate_deployment.py --data kfall`	about 2 min
C port vs Python, synthetic	`python`	under 1 min

Result	Command	Time
	`firmware/test/host/gen_vectors.py --repo . --out build/host/vectors`, then `cmake -S firmware/test/host -B build/host && cmake --build build/host && ctest --test-dir build/host -V`	
C port on real KFall	`gen_vectors.py ... --kfall kfall`, `test_host ... --dump-windows`, then `eval_c_windows.py` (see `firmware/README.md`)	about 1 min
Supervisor tests	`python firmware/tools/test_supervisor.py`	seconds
Emulator end-to-end (needs ESP-IDF)	`firmware/tools/run_qemu_replay.sh esp32` (or `esp32s3`)	about 5 min
Live demo	`firmware/tools/demo.sh` (Docker command in `DEMO.md`)	about 5 min

KFall is not openly licensed: request it from the KFall project page and keep it out of the repository. Tested with Python 3.13, TensorFlow 2.21 and ESP-IDF v5.5.6.

Path	Contents
`phase1_pipeline.py`, `evaluate_deployment.py`	Training pipeline and device-level evaluation
`fall_model_data.h`, `fall_model_int8.tflite`, `fall_cnn.keras`	The model, as the firmware uses it and as trained
`results.md`, `deployment_eval.md`, `fig_*.png`, `deployment_eval.png`	Metrics and figures
`firmware/components/safety_core/`	Portable C signal processing and alert rules
`firmware/main/`	ESP-IDF application, drivers, network link, replay and demo mode
`firmware/test/host/`, `firmware/tools/`	Parity tests, Python device model, supervisor station, emulator and demo scripts
`.github/workflows/firmware.yml`	The seven CI jobs
`README.md`, `firmware/README.md`, `DEMO.md`	Overview, firmware guide with wiring, demo script

Change log

Date	Commit	Change
2026-11-08	`57f603e`	Live demo mode: real-time emulator run with the supervisor page, `demo.sh`, `DEMO.md`, CI demo job
2026-11-08	`cfd5686`	Pipeline re-run with the new threshold rule (0.05 → 0.50; model bit-identical); device-level evaluation report
2026-11-08	`2c7c0d3`	Threshold taken from the middle of the validation plateau; `evaluate_deployment.py`
2026-11-08	`b367ee0`	Man-down, motion-aware vitals, sensor-fault alerts, store-and-forward, acknowledgments, supervisor v2; tensor arena 32 → 16 KB
2026-11-08	`863d524`	Emulator test on the ESP32-S3 as well as the ESP32
2026-11-08	`3a52e97`	Build fix for esp-tflite-micro 1.4.2 (header moved); version pinned to 1.4.x
2026-11-08	`0b61afe`	Phase 2 firmware: C port, drivers, TFLite Micro, Wi-Fi alerts, host parity tests, emulator test, CI
2026-11-08	`857c408`	Pipeline re-run with fixed early stopping (false alarms 5 → 2); README
2026-11-08	`3bb6f6f`	Early stopping on validation loss; `pandas`, `openpyxl` added to requirements
2026-11-08	`acc492c`	Phase 1 zip extracted into the repository
2026-10-12	(upload)	Original Phase 1 outputs: pipeline, model, results (5 false alarms)